CENTRO ESCOLAR CEICAVS — SERVICIO SOCIAL

Documento Técnico del Sistema

Arquitectura, base de datos, integraciones y despliegue del Sistema Escolar CEICAVS

Proyecto:	Sistema Escolar CEICAVS
Rol:	Team Lead / Desarrollador
Responsable:	Francisco Hernandez
Versión:	1.0
Fecha:	Abril 2025

Documento Técnico del Sistema

1. Stack Tecnológico

CAPA	TECNOLOGÍA	PROPÓSITO
Frontend	`React 19` `Vite 6`	SPA con renderizado en cliente, hot module replacement
Estilos	`Tailwind CSS v4` `shadcn/ui`	Sistema de diseño accesible basado en Radix UI
Routing	`React Router v7`	Navegación con loaders y actions (data mode)
Estado servidor	`Apollo Client 4` `GraphQL`	Queries, mutaciones y caché normalizado
i18n	`react-i18next`	Internacionalización (español por defecto)
Autorización frontend	`CASL`	Permisos basados en rol, componente <Can>
IA en navegador	`@huggingface/transformers v4`	Transcripción de audio con Whisper ONNX via Web Worker
Backend	`NestJS 10` `Apollo Server`	API GraphQL code-first
Arquitectura backend	`CQRS`	CommandBus + QueryBus, separación de responsabilidades
Autenticación	`Passport JWT`	Access token (15 min) + refresh token (7 días)
Autorización backend	`CASL` `PoliciesGuard`	Guards a nivel de resolver + verificación en handlers
Base de datos	`PostgreSQL 17` `Prisma 7` `Kysely`	ORM con queries tipadas y driver adapter pg
Almacenamiento	`Cloudinary`	Archivos de audio para transcripciones
LLM primario	`Groq SDK` `llama-3.3-70b-versatile`	Generación de resúmenes de transcripciones
LLM fallback	`Google Gemini` `gemini-2.0-flash`	Respaldo automático ante rate limit de Groq
Monorepo	`Turborepo` `pnpm workspaces`	Compilación incremental y gestión de dependencias
Despliegue	`Vercel` `Render`	Frontend en Vercel, backend en Render

2. Arquitectura General

```
ceicavs/
├── apps/
│   ├── web/          # React SPA (Puerto 5173 local)
│   │   ├── src/
│   │   │   ├── features/       # Módulos por dominio (auth, attendance, blog...)
│   │   │   ├── components/     # Componentes compartidos
│   │   │   ├── generated/      # Tipos GraphQL (auto-generados por codegen)
│   │   │   └── main.tsx
│   │   └── codegen.ts          # Configuración de graphql-codegen
│   │
│   └── api/          # NestJS GraphQL API (Puerto 3001 local)
```

```
│       └── src/
│           ├── modules/        # Módulos verticales (auth, users, attendance...)
│           │   └── [domain]/
│           │       ├── commands/    # Escrituras (CommandHandlers)
│           │       ├── queries/     # Lecturas (QueryHandlers)
│           │       ├── resolvers/   # GraphQL thin resolvers
│           │       ├── repositories/# Implementaciones Prisma
│           │       └── interfaces/  # Repos abstractos + tipos de dominio
│           └── common/
│               └── cqrs/       # BaseCommandHandler, BaseQueryHandler
│
└── packages/
    ├── shared/       # CASL: UserRole, Action, Subject, defineAbilityFor()
    ├── db/           # Prisma 7 + Kysely (query builder tipado)
    └── ui/           # Componentes shadcn/ui compartidos
```

3. Modelo de Datos

Todas las tablas utilizan snake_case via `@map()` / `@@map()` en Prisma. Los registros sensibles implementan **soft delete** mediante el campo `deletedAt`.

User

- id: String (cuid)
- name: String
- email: String (unique)
- password: String (bcrypt)
- role: UserRole (enum)
- createdAt: DateTime
- deletedAt: DateTime?

Group

- id: String (cuid)
- name: String
- createdById: String
- createdAt: DateTime
- deletedAt: DateTime?
- *rel: members, attendances*

Attendance

- id: String (cuid)
- groupId: String
- studentId: String
- date: DateTime
- status: AttendanceStatus
- *enum: PRESENT, ABSENT, LATE, JUSTIFIED*

Post (Blog)

- id: String (cuid)
- title: String
- excerpt: String
- content: String
- status: PostStatus
- authorId: String
- categoryId: String?
- deletedAt: DateTime?

Recording

- id: String (cuid)
- name: String
- audioUrl: String (Cloudinary)
- publicId: String (Cloudinary)
- userId: String
- createdAt: DateTime
- deletedAt: DateTime?
- *rel: transcription*

Transcription

- id: String (cuid)
- recordingId: String (unique)
- fullTranscript: String?
- summary: String?
- keyTakeaways: String[]
- actionItems: String[]
- status: TranscriptionStatus

ENUM	VALORES
UserRole	ADMIN, TEACHER, STUDENT
AttendanceStatus	PRESENT, ABSENT, LATE, JUSTIFIED
PostStatus	DRAFT, PENDING_REVIEW, PUBLISHED, REJECTED
TranscriptionStatus	PENDING, PROCESSING, COMPLETED, FAILED

4. Flujo de Autenticación

1. El usuario envía `email` y `password` a la mutación GraphQL `login`.
2. El servidor verifica la contraseña con `bcrypt.compare()`. Si es correcta, genera `accessToken` (exp: 15 min) y `refreshToken` (exp: 7 días) firmados con JWT.
3. El frontend almacena ambos tokens en `localStorage` y configura el Apollo Link para adjuntar `Authorization: Bearer <accessToken>` en cada petición.
4. `AppBootstrap` rehidrata el contexto de autenticación al recargar la página leyendo el token de `localStorage`.
5. `AbilityProvider` llama a `defineAbilityFor(role)` de `@ceicavs/shared` y publica el objeto CASL en el contexto React.
6. Cuando el `accessToken` expira, el Apollo Link de refresco envía automáticamente la mutación `refreshToken` y retoma la petición original con el nuevo token.

5. Pipeline GraphQL (Code-First → Codegen)

NestJS Backend
@ObjectType, @Resolver

→

`schema.gql`
auto-generado

→

graphql-codegen
apps/web/codegen.ts

→

`src/generated/`
tipos TypeScript exactos

El frontend importa la función `graphql()` generada para definir operaciones tipadas, eliminando la necesidad de declarar tipos manualmente.

6. Arquitectura de Autorización (CASL)

FUENTE ÚNICA DE VERDAD

`packages/shared/src/casl.ts` define los enums `UserRole`, `Action`, `Subject` y la función `defineAbilityFor(role)` usados tanto en el backend como en el frontend.

CAPA	MECANISMO	PROPÓSITO
Resolver (backend)	`@Can(Action.X, Subject.Y)` decorator	Guard de ruta grueso: rol vs. recurso
Handler (backend)	`ability.can(Action.X, Subject.Y)`	Verificación fina: ownership, condiciones
Rutas (frontend)	`<RequireAbility>` component	Redirección si el usuario no tiene permiso
UI (frontend)	`<Can I={Action.X} a={Subject.Y}>`	Oculta botones y controles no permitidos

7. Integraciones Externas

SERVICIO	TECNOLOGÍA	PROPÓSITO	UBICACIÓN
Transcripción de audio	Transformers.js + Whisper ONNX	Audio → texto directamente en el navegador via Web Worker. Sin enviar datos a servidores externos.	Frontend
Almacenamiento de audio	Cloudinary REST API	Subir y almacenar archivos de audio. Se almacena `audioUrl` y `publicId` en la base de datos.	Frontend
Resumen LLM (primario)	Groq SDK — `llama-3.3-70b-versatile`	Genera resumen estructurado (summary, keyTakeaways, actionItems) a partir de la transcripción.	Backend
Resumen LLM (fallback)	Google Gemini — `gemini-2.0-flash`	Respaldo automático cuando Groq retorna HTTP 429 (rate limit).	Backend
YouTube Metadata	yt-dlp + YouTube Data API	Obtener metadatos y enlace de descarga de videos de YouTube para la herramienta educativa.	Backend

8. Arquitectura del Despliegue

```
                    INTERNET
                       |
        ┌──────────────┴──────────────┐
        |                             |
[Usuario]                     [Usuario]
HTTPS: Vercel                 HTTPS: Vercel
        |                             |
┌───────▼─────────────────────────────▼──────┐
|          Vercel CDN (Edge Network)         |
|          apps/web  (React SPA)             |
└────────────────┬────────────────────────────┘
                 | GraphQL HTTPS
┌────────────────▼────────────────────────────┐
|         Render.com (Web Service)            |
|         apps/api  (NestJS + Apollo)         |
└──────┬──────────────────┬──────────────────┘
       |                  |
┌──────▼──────┐    ┌──────▼──────┐
| PostgreSQL  |    |  Cloudinary |
|  (Render    |    |  (audio files)|
|   Managed)  |    └─────────────┘
└─────────────┘

       |
┌──────▼──────────────────────────┐
|  Groq API / Google Gemini API   |
|  (generación de resúmenes)      |
└─────────────────────────────────┘
```

9. Variables de Entorno

VARIABLE	UBICACIÓN	DESCRIPCIÓN
`DATABASE_URL`	Backend	Cadena de conexión PostgreSQL

VARIABLE	UBICACIÓN	DESCRIPCIÓN
`JWT_ACCESS_SECRET`	Backend	Clave para firmar access tokens
`JWT_REFRESH_SECRET`	Backend	Clave para firmar refresh tokens
`GROQ_API_KEY`	Backend	API key del servicio Groq
`GEMINI_API_KEY`	Backend	API key de Google Gemini (fallback)
`CLOUDINARY_CLOUD_NAME`	Frontend	Identificador de la cuenta Cloudinary
`CLOUDINARY_UPLOAD_PRESET`	Frontend	Preset de carga pública de Cloudinary
`VITE_GRAPHQL_URL`	Frontend	URL del endpoint GraphQL del backend